Predicting Used Car Value Retention Ratio Using Multi Source Data and Machine Learning

Anthony Wert

CMPSC 445

Project 1

1. Project Description

This project aimed to predict how much of a vehicles original value is retained when it is listed for sale as a used car. I collected listings from Craigslist and OfferUp, combined the data, gathered original MSRP information, and compared three regression algorithms. I also looked at which vehicle characteristics were most useful for predicting depreciation.

Value retention ratio = current used vehicle listing price / original MSRP. For example, a vehicle advertised for $15000 with an original MSRP of $30000 has a retention ratio of 0.50, or 50%. The ratio measures the asking price relative to the original new vehicle price. It does not measure the actual transaction price or account for inflation.

The project focuses on three makes and three models from each make: Ford (F-150, Explorer, Escape), Chevrolet (Silverado 1500, Equinox, Malibu), and Toyota (Camry, Corolla, RAV4). The observed model years range from 2005 through 2026

2. Data Collection

I used separate Python programs to collect listings from the two websites. For OfferUp, I used Selenium to navigate vehicle searches and BeautifulSoup to extract information from individual listings. The Craigslist scraper searched several regional marketplaces and collected similar vehicle attributes. Both programs saved the raw records and a separate preview of initially usable listings. Along the way, I had to deal with pagination, changes in page layout, duplicate advertisements, and missing specifications.

The scrapers collected 902 raw OfferUp records and 869 raw Craigslist records. After initial validation and selecting the required number of listings per model, each source contributed 675 usable listings, for a combined total of 1350. This exceeds the requirement of at least 500 listings from each website before integration.

Source	Raw parsed listings	Initially usable listings
Craigslist	869	675
OfferUp	902	675
Total	1771	1350

The scraped attributes include make, model, model year, asking price, mileage, trim, vehicle condition, fuel type, transmission, drivetrain, body style, exterior color, title status, location, VIN when available, and seller description. This provided more than ten original attributes rather than just engineered features. Some optional fields were often missing, though, and information such as engine details and interior color was not consistently available.

The two sites also worked differently during collection. OfferUp allowed broad searches using a large distance setting, while Craigslist required searching individual regional marketplaces. I kept

the original listing files in data/raw/ so they remain separate from the data produced during cleaning.

3. Data Preprocessing and Integration

I used pandas to load both datasets and bring their attributes into a consistent set of columns. I converted price, mileage, and model year to numeric values, removed records with invalid essential data or mismatched models, and checked listing URLs and valid VINs for duplicates. I also standardized categories that the websites labeled differently, including fuel type, drivetrain, condition, body style, and transmission.

The initial cleaning left 1345 records. Five listings referred to related but different vehicles, including Camry Solara and Corolla Cross, so I removed them. MSRP matching and the final quality checks excluded another 140 records because of suspicious asking prices, possible financing advertisements, or other concerns. The final modeling dataset contained 1205 listings, with 612 from Craigslist and 593 from OfferUp.

Stage	**Records**
Initially usable listings from both sources	1350
After integration and initial cleaning	1345
Excluded before model training	140
Final model-ready dataset	1205

Among the final modeling records, the median asking price was $7000 and the median mileage was 135800 miles. The average asking price was approximately $9819, and average mileage was approximately 132809 miles. These statistics describe the filtered data used for modeling, rather than all of the advertisements initially collected.

Some raw listings contained extreme values. One entry in the integrated dataset, for example, had an impossible mileage reading of more than one billion miles. That value distorted the original mean and showed why the data needed to be filtered before interpreting the statistics. For the report, I focus on the filtered records. I also ran a separate duplicate review that found listings with the same year, make, model, price, and mileage. I did not remove all of them automatically because matching specifications do not necessarily mean two advertisements are for the same vehicle.

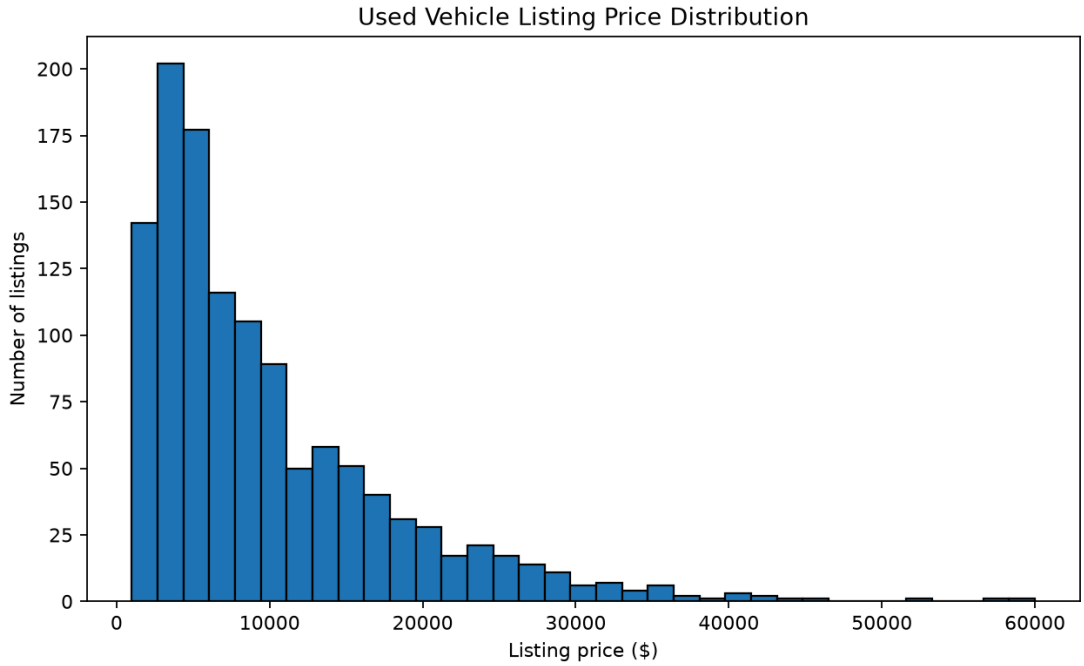


Figure 1. Distribution of asking prices in the modeling dataset.

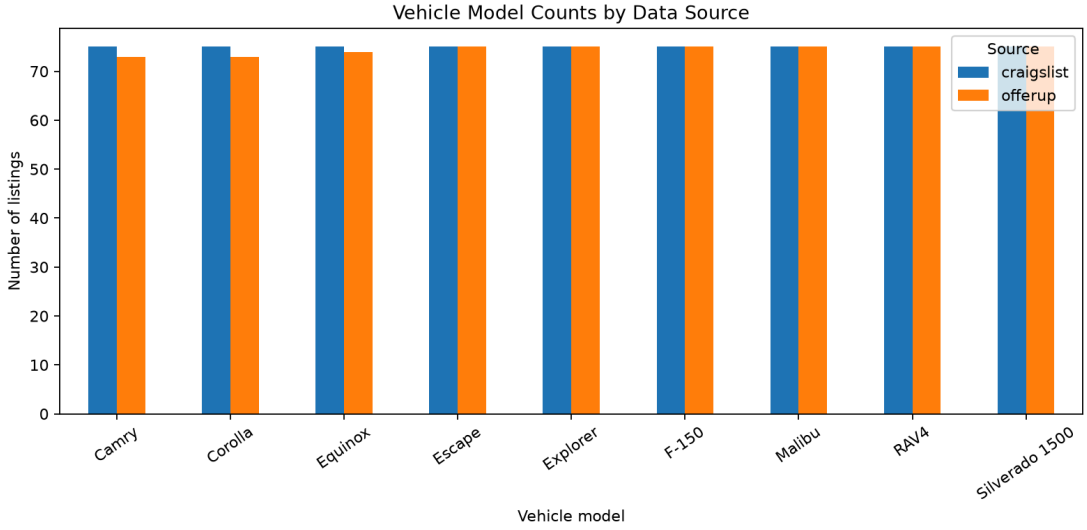


Figure 2. Listing counts by selected vehicle model and source.

4. Historical MSRP Collection and Matching

To calculate value retention, I needed the original MSRP for the vehicles in the dataset. I wrote a separate collector to retrieve configuration level prices from historical specification pages on The Car Connection. The resulting reference table contains 3454 configuration rows and covers all 187 make, model, and model year combinations present in the integrated listings. I saved the source page URL with each entry.

I matched the used listings against this reference using model year, make, model, and any available trim or drivetrain information. When two records gave conflicting MSRP values for the same configuration, I left that configuration out of the normal matching process. I also recorded

how each price was matched. Of the 1205 records used for modeling, 388 received trim and drivetrain approximations, 331 received trim approximations, and 486 used a base MSRP for the model year.

When the exact original configuration was unclear, the fallback used an available lower priced configuration from the same model year. This made the matching process consistent, but it also introduced uncertainty. A more expensive trim could have had a higher original MSRP, meaning the fallback would make its retention ratio appear higher than it really was. Since actual selling prices were unavailable as well, the ratios should be viewed as estimates based on advertised prices and approximate original MSRP values.

5. Feature Engineering

The main numerical predictors included vehicle age and mileage. I calculated vehicle age by subtracting model year from the analysis year, with a minimum of zero. I also used patterns in seller descriptions to create three binary features: one owner claims, mentions of accidents, and mentions of a clean title. These features describe the wording of an advertisement, not verified vehicle history. For example, the accident feature can also detect phrases such as "no accidents," so it does not confirm that an accident occurred.

The categorical inputs included make, model, trim, condition, fuel type, transmission, drivetrain, body style, exterior color, title status, and source. Missing numeric values were filled using the median calculated from training data, and missing categorical values were labeled "unknown". I standardized the numeric values with StandardScaler and used OneHotEncoder with min_frequency=10 and handle_unknown="infrequent_if_exist" for the categorical values. Grouping infrequent categories was especially helpful for trim, which had many different labels. These operations were kept inside the scikit learn pipelines so the test data was not used to fit the preprocessing steps.

I left listing price and original MSRP out of the predictors because those values were already used to calculate the target ratio. Including them would give the model information about the answer instead of relying on the vehicle characteristics.

6. Model Development

I compared Linear Regression, Random Forest Regression, and Gradient Boosting Regression using scikit learn. Linear Regression provided a simple baseline. Random Forest can capture more complicated relationships by combining predictions from many decision trees, while Gradient Boosting builds trees in sequence to improve on earlier prediction errors.

I split the filtered dataset into 80% training data and 20% testing data, giving 964 training records and 241 testing records. The split used random_state=42 and was stratified by vehicle model. All three algorithms used the same preprocessing approach and test set so their results could be compared fairly. Random Forest used 150 trees and min_samples_leaf=2. I evaluated performance on one held out test set rather than running a full cross validation or parameter search.

7. Evaluation Results

I measured model performance with mean absolute error (MAE), root mean squared error (RMSE), and R squared (R2). Lower MAE and RMSE indicate smaller prediction errors, while a higher R2 indicates that the model explains more of the variation in the test data. These errors are measured in retention ratio units, rather than dollars.

Model	MAE	RMSE	R2
Linear Regression	0.0918	0.1262	0.7441
Random Forest	0.0827	0.1198	0.7697
Gradient Boosting	0.0815	0.1172	0.7794

Gradient Boosting had the best results on the test set, with an MAE of 0.0815, RMSE of 0.1172, and R2 of 0.7794. The MAE corresponds to an average absolute error of about 8.15 percentage points in predicted retention. Random Forest was close behind, while Linear Regression performed less accurately on this split. The difference between Gradient Boosting and Random Forest was relatively small, so the results do not establish that Gradient Boosting would be better on every new dataset.

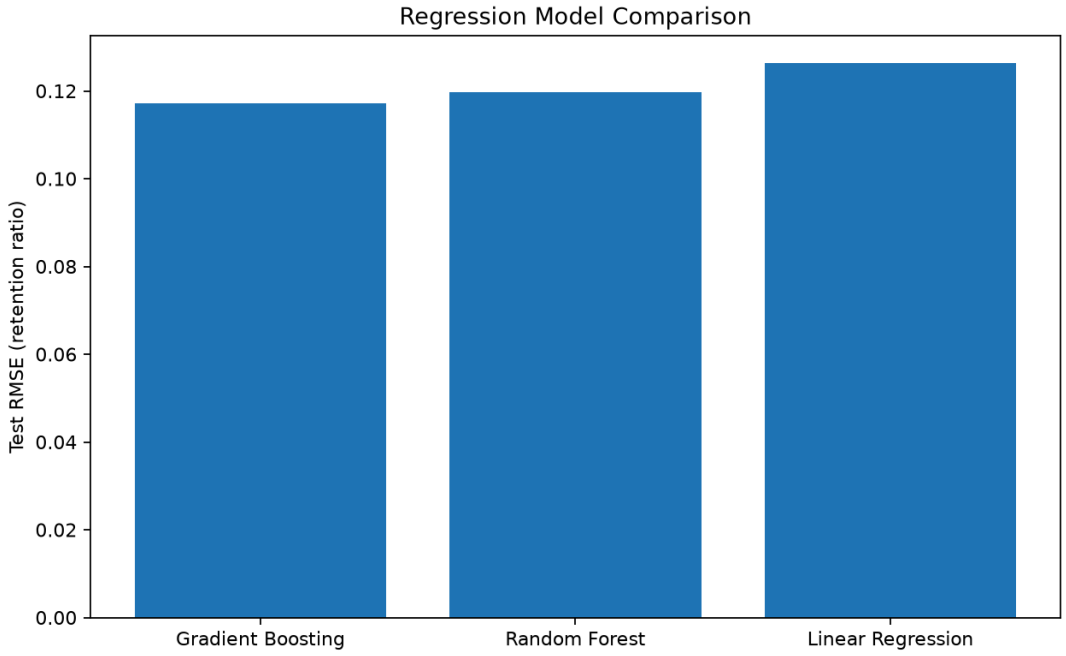


Figure 3. Comparison of regression models based on test set prediction error.

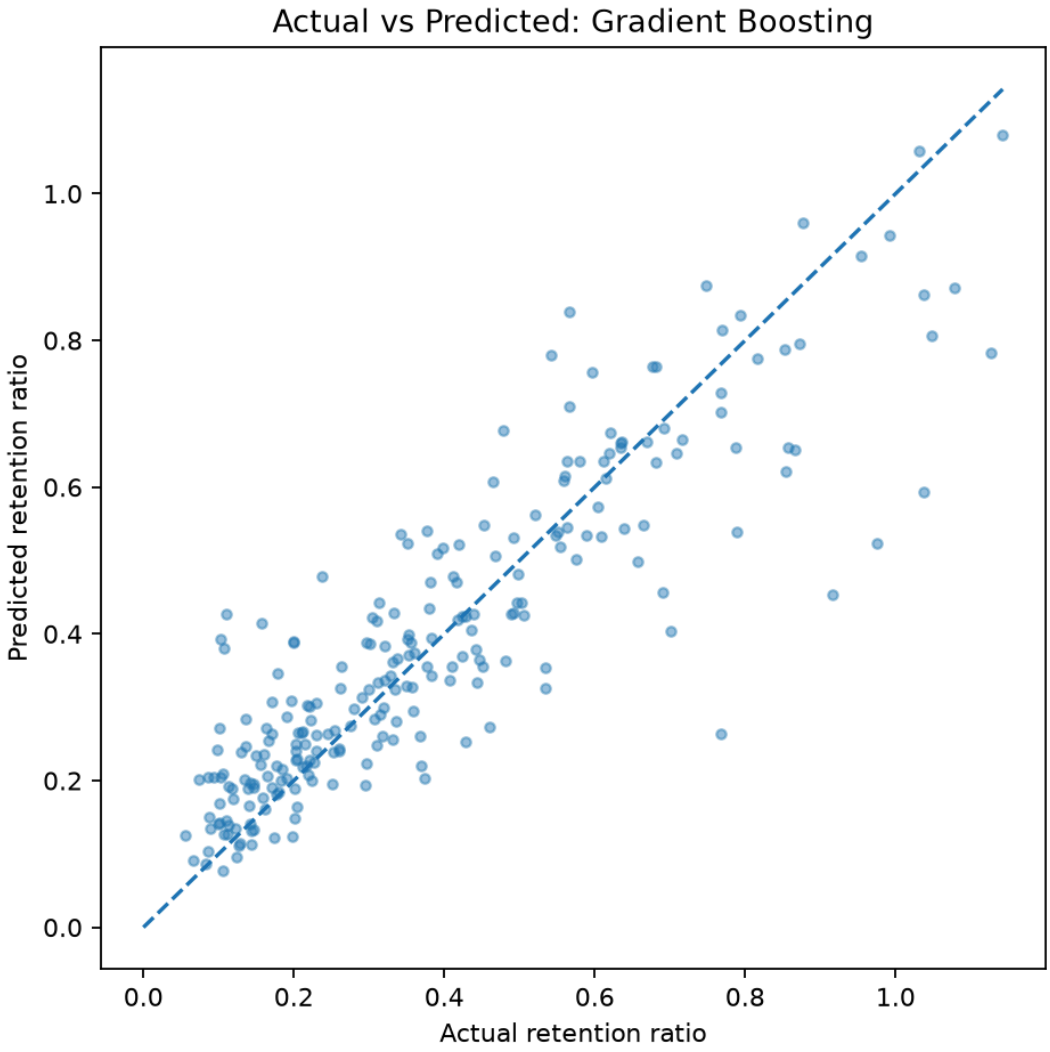


Figure 4. Observed versus predicted retention ratios for the best model.

8. Depreciation Analysis and Feature Ranking

I used permutation importance to see which original inputs were most useful to the fitted Random Forest model. Each feature was shuffled five times in the held out test data, and I recorded how much the R2 score decreased on average. This made it possible to rank attributes such as make and model as whole features, rather than treating every encoded category separately.

Vehicle age ranked first, lowering test R2 by about 0.550 when shuffled. Mileage was next at about 0.281, followed by make at 0.145 and model at 0.092. The remaining features had much smaller individual scores. These values show how useful each feature was for prediction, not its percentage contribution or a proven causal effect. Age and mileage are also related, so they can share some of the same predictive information.

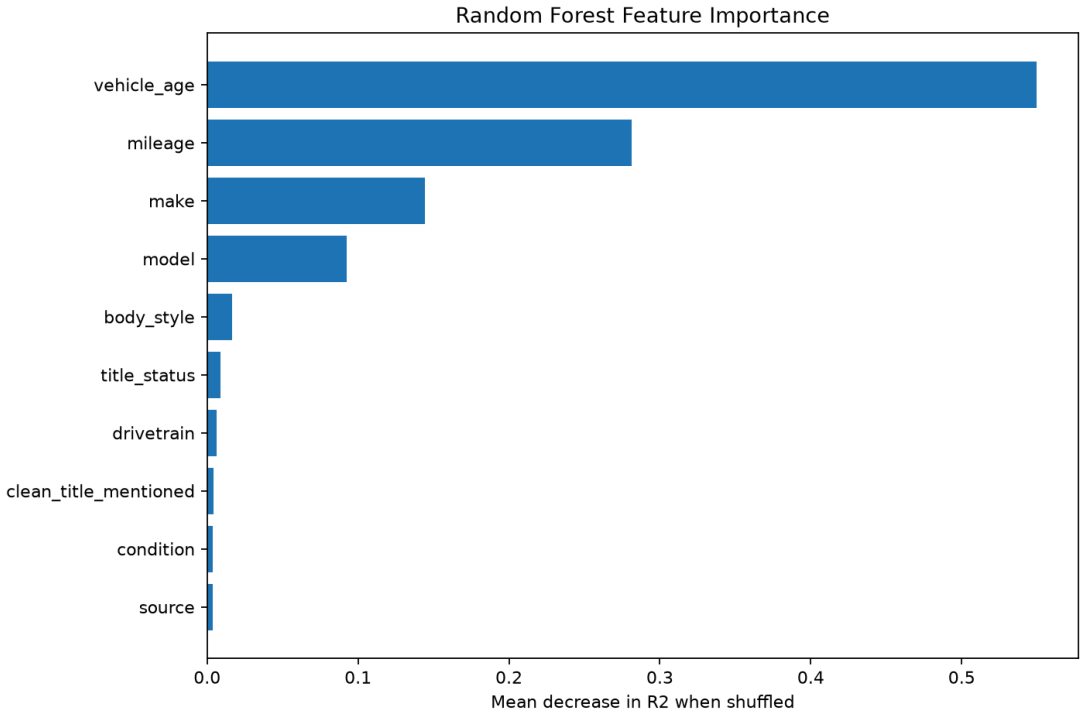


Figure 5. Permutation feature importance for original input attributes.

The age analysis showed that median retention generally declined as vehicles got older. For example, median retention was about 0.894 for vehicles one year old, 0.376 for vehicles ten years old, and 0.160 for vehicles sixteen years old. The pattern was not perfectly smooth because different age groups contained different makes, models, mileages, conditions, and numbers of listings. Even with those variations, older vehicles generally retained less of their original MSRP.

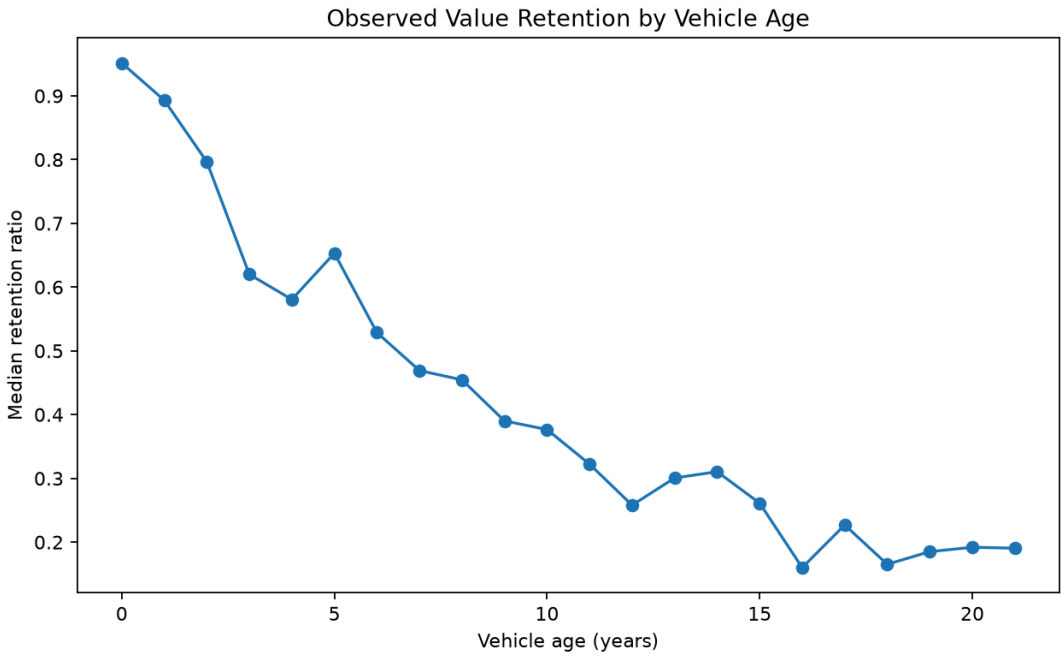


Figure 6. Median observed retention ratio by vehicle age.

9. Discussion and Limitations

The results point to vehicle age and mileage as the strongest indicators of advertised value retention. Make and model also contribute because vehicles differ in demand and original pricing. The tree based models performed better than Linear Regression, suggesting that the relationship between vehicle characteristics and depreciation is not entirely linear.

There are some important limitations to these findings. The dataset uses asking prices rather than confirmed sale prices, and many MSRP matches are estimates rather than exact original window sticker prices. I balanced collection by make and model, but not by year; 245 of the 1205 modeling records came from 2005. The websites also differed in their default values and the detail they provided, and some listings lacked VINs or complete specifications. The duplicate review found advertisements that might have been posted more than once, but the available information was not enough to verify every case. Finally, I evaluated the models using one train and test split, so results could differ with newer listings or other websites.

A few parts of the project took more work than expected. On OfferUp, I had to search for specific model years to find enough listings. Craigslist initially returned zero candidates because the website used a different listing link format than the scraper expected. The historical MSRP collector also occasionally encountered human verification pages. Fixing these issues took several rounds of testing. Keeping the raw records separate from the cleaned data was useful because I could make corrections without losing the original results.

10. Conclusion

This project brought together vehicle data collection, cleaning, historical MSRP matching, feature engineering, and regression modeling in one pipeline. After the final filtering, I had 1205 records, which met the minimum dataset requirement. Gradient Boosting gave the strongest results of the three tested algorithms on the held out test set, with an R2 of approximately 0.779.

The main finding was that vehicle age and mileage were strongly associated with lower value retention, while make and model provided additional information. I also learned that collecting enough listings was only the beginning. I still had to identify possible duplicates, standardize categories across websites, remove misleading prices, and connect each vehicle to an appropriate historical MSRP before interpreting the results. One of the biggest lessons was that a program running successfully does not necessarily mean its data is reliable. Future improvements could include exact MSRP values based on VIN, verified sale prices, better duplicate detection, and cross validation or additional model tuning.

11. AI Usage Statement

I used ChatGPT as an assistant during this project. It helped draft and revise parts of the Python scraping, preprocessing, MSRP matching, exploration, and machine learning code. It also helped troubleshoot scraper errors, suggest data quality checks, and compared the progress with assignment requirements. I also used Claude to critique the implementation and identify possible weaknesses. Throughout the project, I worked in PyCharm to run and test the programs, examine their outputs, and identify problems that needed to be addressed. I tested different approaches to collecting vehicle listings, evaluated the quality and completeness of the collected data, and made decisions about which changes to implement. I also reviewed the preprocessing and machine learning results, checked the generated datasets and visualizations, and organized the final files and GitHub repository. The resulting code and conclusions should be understood as work developed with regular AI assistance, but with a human developer responsible for testing which implementations were best, and deciding how to proceed in each stage.

12. Data Sources and Repository

Vehicle listing sources: Craigslist (https://www.craigslist.org/) and OfferUp (https://offerup.com/). Historical MSRP source: The Car Connection specifications pages (https://www.thecarconnection.com/). Each collected MSRP row records its specific source URL in the reference dataset. The project repository contains the scripts, raw listing files, integrated and model ready CSVs, MSRP reference, analysis tables, visualizations, and this report

GitHub repository: https://github.com/ACW5847/CMPSC_455_Project-1